

Cache-Aware and GPU-Accelerated Charge Deposition for Electrostatic Particle-In-Cell Simulations

Elhadj Hocine
Independent Researcher
Email: hocineelhadj@gmail.com

Abstract—Charge deposition is a dominant cost in explicit Particle-In-Cell (PIC) plasma simulations because it performs irregular, scatter-oriented updates to a structured grid. This work presents a reproducible two-dimensional electrostatic PIC framework and evaluates Nearest Grid Point (NGP), Cloud-In-Cell (CIC), Triangular-Shaped Cloud (TSC), and Esirkepov charge-conserving deposition under cache-aware CPU optimizations. We compare Array-of-Structures (AoS) and Structure-of-Arrays (SoA) layouts, optional cell sorting, and OpenMP parallelization with thread-local grid privatization. Benchmarks measure sort and deposit time separately, effective memory bandwidth, charge conservation error, energy drift, spectral grid noise, and CPU strong scaling on particle counts from 10^4 to 10^6 . A Langmuir-wave validation confirms scheme-independent oscillation frequencies within 5% across all four deposition methods. GPU deposition kernels are implemented but left for future performance evaluation. Results show AoS and SoA achieve comparable deposit throughput, sorting adds overhead when timed per call, CIC offers the best accuracy–performance trade-off, and OpenMP privatization yields modest speedups before merge cost dominates.

Index Terms—Particle-In-Cell, charge deposition, high performance computing, cache optimization, plasma simulation

I. Introduction

Particle-In-Cell (PIC) methods combine Lagrangian macroparticles with Eulerian field representations and are widely used in kinetic plasma simulation [1]–[3]. Each explicit timestep performs charge deposition, field solve, field gather, and particle push. Among these stages, charge deposition is frequently a bottleneck because it scatters particle contributions into a grid with irregular memory access patterns [4]. Prior studies have shown that shape-function choice controls grid noise and stability [5], while charge-conserving deposition is essential for long-time accuracy [6], [7].

This work presents a performance-oriented evaluation of deposition algorithms and cache-aware CPU implementations in a reproducible 2D electrostatic PIC framework. This paper reports CPU microbenchmarks and physics validation; GPU throughput evaluation is planned once CUDA hardware is available. We study NGP, CIC, TSC, and Esirkepov charge-conserving deposition [6], along with

CUDA deposition kernels (atomic and privatized-tile variants) are implemented and buildable with BUILD_CUDA=ON, but no GPU performance numbers appear in this version; all reported benchmarks and validation use CPU backends.

particle sorting, AoS versus SoA layouts, and OpenMP parallelization with thread-local grid buffers. CUDA deposition kernels (atomics and privatized tiles) are included in the codebase but not benchmarked in this study.

Our contributions are:

- A reproducible 2D electrostatic PIC reference code with FFT-based Poisson solve and unified deposition interface.
- A systematic comparison of four deposition schemes with charge, energy, and spectral-noise diagnostics.
- Cache-aware CPU study separating sort cost from deposit-kernel throughput for AoS/SoA layouts.
- Langmuir-wave validation and OpenMP privatization benchmarks on Apple Silicon.

II. Mathematical Background

The continuous charge density is

$$\rho(\mathbf{x}) = \sum_p q_p S(\mathbf{x} - \mathbf{x}_p), \quad (1)$$

where S is a compact shape function. Deposited charge must satisfy global conservation,

$$\int \rho(\mathbf{x}) dV = \sum_p q_p, \quad (2)$$

otherwise nonphysical fields and instability can arise [8]. We report conservation error normalized by $\sum_p |q_p|$ to handle charge-neutral plasmas.

In the electrostatic limit,

$$\nabla^2 \phi = -\frac{\rho}{\epsilon_0}, \quad \mathbf{E} = -\nabla \phi. \quad (3)$$

Energy is tracked as $W = \sum_p \frac{1}{2} m_p v_p^2 + \frac{1}{2} \int |\mathbf{E}|^2 dV$.

Spectral noise is quantified by the high- k power fraction of the deposited density field after a 2D FFT, with wavenumbers above one-third of the Nyquist limit classified as noise.

A. Langmuir Wave Validation Model

We validate linear physics with a cold Langmuir-wave test. Macroparticles carry charge $q_p = -1/N_p$ on a uniform 1D-like slab at fixed y , with a neutralizing background density $\rho_{bg} = 1/(L_x L_y)$ added after each

TABLE I
Deposition stencil characteristics in 2D.

Scheme	Nodes	Writes/particle	Conservation
NGP	1	1	Point assignment
CIC	4	4	Not exact; low error
TSC	9	9	Not exact; lowest noise
Esirkepov	$\mathcal{O}(L)$	$\mathcal{O}(L^2)$	Exact along trajectory

deposition. A sinusoidal velocity perturbation $v_x(x) = \delta v \sin(kx)$ is imposed at initialization with $k = 2\pi m/L_x$. In normalized units ($m = 1$), the macro-particle plasma frequency is

$$\omega_{p,\text{macro}} = \sqrt{\frac{N_p}{L_x L_y} \left(\frac{1}{N_p}\right)^2} = \frac{1}{\sqrt{N_p L_x L_y}}. \quad (4)$$

The dominant oscillation frequency ω_{meas} is extracted from field-energy time series by discrete Fourier analysis. The primary validation metric is inter-scheme agreement: all deposition schemes should yield the same ω_{meas} within 5%. We additionally report $\omega_{\text{meas}}/\omega_{p,\text{macro}}$ to document the systematic offset introduced by the 2D field-energy diagnostic and discrete solver; this ratio is not used as a pass/fail criterion.

III. Related Work

Early PIC studies established that spatial grid resolution and shape-function choice control aliasing noise and numerical heating [5], [9]. Brackbill and Barnes showed that discrete charge non-conservation can destabilize long-time integrations [8], motivating exact deposition schemes. Esirkepov's trajectory-based method provides rigorous charge conservation for arbitrary shape factors [6], while Eastwood's virtual-particle formulation extended conservative deposition to electromagnetic PIC [10]. Harvey et al. demonstrated charge-conserving deposition on hybrid CPU-GPU systems [7].

Recent work has focused on adapting PIC algorithms to emerging architectures. Viereck et al. surveyed memory-access and parallelization challenges for deposition on modern hardware [4]. Wilke et al. reported GPU-accelerated PIC strategies for large-scale supercomputers [11]. Our contribution differs by providing a unified, reproducible 2D reference implementation that separates sort cost from deposit-kernel throughput and couples CPU microbenchmarks with charge, energy, spectral-noise, and Langmuir-wave validation under a common runtime interface.

IV. Deposition Methods

Charge deposition maps N_p macroparticles onto an $N_x \times N_y$ grid by scattering weighted contributions into ρ . Each scheme trades stencil width, arithmetic cost, and spectral smoothness. Table I summarizes the per-particle grid touches in our 2D implementation.

L is the number of grid cells crossed by the particle trajectory in one dimension.

A. Nearest Grid Point (NGP)

NGP assigns each macroparticle charge q_p to the nearest grid node (j_x, j_y) . Given fractional cell coordinates (w_x, w_y) from the particle position,

$$j_x = \begin{cases} i_x + 1 & w_x \geq 0.5 \\ i_x & \text{otherwise} \end{cases}, \quad (5)$$

with analogous rule for j_y , and update

$$\rho_{j_x, j_y} += \frac{q_p}{\Delta x \Delta y}. \quad (6)$$

NGP performs one read-modify-write per particle and is the fastest scheme, but its discontinuous shape function injects the highest grid noise.

B. Cloud-In-Cell (CIC)

CIC distributes charge to the four nodes of the host cell using separable linear weights. For one dimension,

$$w_1 = 1 - \xi, \quad w_2 = \xi, \quad \xi = \frac{x_p - x_i}{\Delta x}, \quad (7)$$

and in 2D,

$$\rho_{i,j} += \frac{q_p}{\Delta x \Delta y} \sum_{\alpha, \beta \in \{0,1\}} w_x^{(\alpha)} w_y^{(\beta)}, \quad (8)$$

where $w_x^{(0)} = 1 - w_x$, $w_x^{(1)} = w_x$, and similarly for y . CIC is the standard production choice: four grid writes per particle with substantially lower high- k noise than NGP.

C. Triangular-Shaped Cloud (TSC)

TSC uses a three-point quadratic B-spline per dimension, touching nine nodes in 2D. With normalized coordinate $\xi = x_p/\Delta x$ and base index $i_0 = \lfloor \xi - 0.5 \rfloor$, let $x = \xi - i_0$. The weights are

$$w_0(x) = \frac{1}{2}(1.5 - x)^2, \quad (9)$$

$$w_1(x) = 0.75 - (x - 1)^2, \quad (10)$$

$$w_2(x) = \frac{1}{2}(x - 0.5)^2. \quad (11)$$

The 2D deposit is the tensor product $w_x^{(d_x)} w_y^{(d_y)}$ over $d_x, d_y \in \{0, 1, 2\}$. TSC reduces spectral noise relative to CIC at the cost of nine scatter writes and additional weight arithmetic.

D. Esirkepov Charge-Conserving Deposition

Esirkepov's scheme deposits charge along the particle trajectory so that the discrete charge density update is exactly conservative [6]. For a particle moving from $\mathbf{x}_0$ to $\mathbf{x}_1 = \mathbf{x}_0 + \mathbf{v} \Delta t$ over one timestep, 1D trajectory weights W_i are accumulated by marching through crossed cell boundaries:

$$W_i += \frac{x_{\text{next}} - x_{\text{curr}}}{\Delta x}, \quad (12)$$

where x_{next} is clipped to the next cell face or the endpoint. In 2D we form separable weights W_i^x and W_j^y and deposit only over the sparse outer product of nonzero supports:

$$\rho_{i,j} += \frac{q_p}{\Delta x \Delta y} W_i^x W_j^y. \quad (13)$$

Our implementation stores only cells touched by the trajectory, giving $\mathcal{O}(L_x L_y)$ writes per particle where L_x and L_y are the number of cells crossed in each dimension.

V. Performance Engineering

Charge deposition is memory-bound on CPU because particles are read sequentially while grid writes are irregular and often non-local. We apply three complementary optimizations: cell sorting for spatial locality, SoA layout for vector-friendly particle reads, and OpenMP grid privatization to avoid atomic contention.

A. Particle Sorting

Before deposition, particles are sorted by cell index $c = i_y N_x + i_x$, computed from periodic-wrapped coordinates. For AoS, `std::sort` reorders the particle array in place; for SoA, an index permutation is sorted and attributes are gathered into a new buffer. Sorting costs $\mathcal{O}(N_p \log N_p)$ but clusters particles that write to nearby grid cells, improving cache reuse during the scatter loop. In timed microbenchmarks, sort and deposit are measured separately so that locality benefits are not conflated with reordering overhead.

B. AoS vs SoA

AoS stores (x, y, v_x, v_y, q, m) in a 48-byte struct per particle; SoA stores each attribute in a separate contiguous array (40 bytes read per particle during deposition). SoA enables unit-stride access to positions and charges, which is favorable when the inner loop touches only a subset of attributes. AoS can be faster to sort because reordering a single array avoids the SoA gather pass. Our benchmarks compare both layouts under identical schemes and thread counts.

C. OpenMP Privatization

Parallel CPU deposition partitions the particle index range across T threads. Each thread allocates a private copy of the full ρ grid, deposits its particle subset locally, then merges into the global grid inside an omp critical region:

$$\rho_k^{\text{global}} += \sum_{t=0}^{T-1} \rho_k^{\text{local},t}. \quad (14)$$

This avoids per-cell atomics that serialize cache lines under high thread counts. The trade-off is $T \times$ temporary grid storage and a merge pass whose cost grows with grid size and thread count.

D. GPU Strategies (Future Work)

CUDA kernels implementing atomic and privatized tile deposition are compiled when `BUILD_CUDA=ON`. The atomic variant performs direct scatter updates with hardware atomics; the privatized variant assigns each thread block a tile of the grid, accumulates locally, and merges tiles afterward. These backends share the same `DepositionConfig` interface as the CPU path but are not benchmarked in this study.

VI. Implementation

The reference code is implemented in C++17 with OpenMP and FFTW 3.3. Periodic boundaries are enforced by wrapping particle positions and grid indices modulo (N_x, N_y) . The unit domain is $[0, 1]^2$ with cell volume $\Delta x \Delta y$.

A. Software Architecture

Three executables are provided:

- `pic_sim`: full electrostatic PIC timestep loop for interactive runs.
- `pic_benchmark`: sweeps the deposition parameter matrix and writes CSV results.
- `pic_validation`: runs charge, spectral-noise, and Langmuir-wave diagnostics.

Deposition kernels share a `DepositionConfig` structure selectable at runtime by scheme, particle layout (AoS/SoA), sorting flag, thread count, and backend (CPU, GPU atomics, or GPU privatized). Each scheme exposes paired AoS and SoA entry points plus OpenMP-parallel variants for $T > 1$.

B. Timestep Cycle

The explicit leapfrog cycle is scheme-dependent during deposition. For NGP, CIC, and TSC:

- 1) Particle push: $\mathbf{x}_p += \mathbf{v}_p \Delta t$ with periodic wrap.
- 2) Charge deposition at the new particle position.
- 3) Poisson solve: FFT-based spectral solve computes ϕ and $\mathbf{E}$.
- 4) Field gather: CIC interpolation of $\mathbf{E}$, then $\mathbf{v}_p += -(q_p/m_p)\mathbf{E} \Delta t$.

For Esirkepov, deposition must use the pre-push position and the displacement $\mathbf{v}_p \Delta t$:

- 1) Charge deposition along $[\mathbf{x}_p, \mathbf{x}_p + \mathbf{v}_p \Delta t]$.
- 2) Particle push.
- 3) Poisson solve and field gather as above.

This ordering preserves trajectory-based charge conservation.

C. Diagnostics and Reproducibility

After deposition, we compute charge conservation error

$$\epsilon_Q = \frac{\left| \sum_{i,j} \rho_{i,j} \Delta x \Delta y - \sum_p q_p \right|}{\sum_p |q_p|}, \quad (15)$$

spectral noise as the fraction of FFT power above one-third of the Nyquist wavenumber in each dimension, and total energy $W = W_{\text{kin}} + W_{\text{field}}$. Benchmark output is written as CSV under `results/`; `scripts/plot_results.py` converts these files to the figures in this paper.

D. Data and Code Availability

The complete source code, build scripts, and archived benchmark CSVs are distributed with this project. Figures and summary tables regenerate from the archived data in `data/benchmarks/` via `scripts/plot_results.py`. GPU deposition benchmarks will be added in a future revision once CUDA hardware is available.

VII. Experimental Methodology

A. Benchmark Matrix

Microbenchmarks sweep a full factorial over:

- Particle counts $N_p \in \{10^4, 10^5, 10^6\}$.
- Grid sizes 64^2 , 128^2 , and 256^2 on the unit square.
- Schemes: NGP, CIC, TSC, Esirkepov.
- Layouts: AoS and SoA.
- Sorting: enabled and disabled.

Each case uses 5 timed repeats after 3 warmup depositions. Particles are initialized from a uniform Maxwellian with fixed seed 42. Esirkepov microbenchmarks use $\Delta t = 0.01$ to form trajectory weights. CPU scaling cases fix $N_p = 10^5$, grid 128^2 , CIC, SoA, no sorting, and thread counts $T \in \{1, 2, 4, 8\}$ with privatized OpenMP buffers. Energy-drift simulations run 500 steps for all schemes at $N_p = 10^5$ on a 128^2 grid with sorting enabled.

B. Timing Protocol

Timed deposition separates sort cost from kernel cost. When sorting is enabled, one sort is timed, then the deposit kernel is averaged over repeats iterations on the sorted particle order. Throughput is reported as N_p/t_{deposit} and effective bandwidth as $(B_{\text{read}} + B_{\text{write}})/t_{\text{deposit}}$, where B_{read} counts particle record bytes and B_{write} counts stencil updates plus one grid clear. Sort time is excluded from throughput and bandwidth so that locality effects can be compared independently.

C. Physics Validation

Langmuir-wave validation uses $N_p = 200$ particles on a 64^2 grid with $\Delta t = 0.002$ and 16384 timesteps. A sinusoidal velocity perturbation of amplitude 0.05 is imposed at mode number $m = 1$. After each deposition, a neutralizing background $\rho_{\text{bg}} = 1/(L_x L_y)$ is added. Field energy is recorded over the final three-quarters of the run and analyzed by discrete Fourier transform. Pass criterion: measured frequency spread across all four schemes $\leq 5\%$. We also report $\omega_{\text{meas}}/\omega_{p,\text{macro}}$ to document the systematic offset from the 1D cold-plasma estimate in this 2D diagnostic setup.

Energy-drift validation uses separate 500-step full-PIC simulations at $N_p = 10^5$ on a 128^2 grid.

TABLE II
Benchmark platform configuration.

Component	Specification
CPU	Apple M1 Pro
Cores	8 (OpenMP threads: 1, 2, 4, 8)
OS	macOS 26.5.1
Compiler	Apple clang 21.0.0, C++17, -O3
Libraries	FFTW 3.3.11, libomp 22.1.7

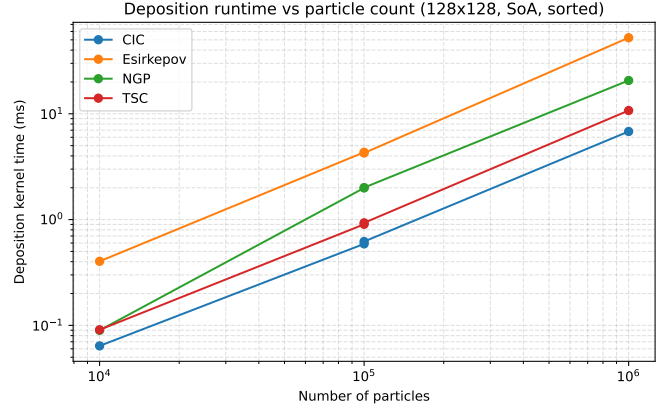


Fig. 1. Deposit-kernel runtime vs. particle count (128×128 grid, SoA, sorted).

D. Hardware and Scope

All benchmarks were built with `cmake -DCMAKE_BUILD_TYPE=Release` and executed on the platform summarized in Table II. GPU evaluation is deferred to future work on CUDA hardware; CUDA kernels compile with `BUILD_CUDA=ON` but are not timed in this study. All reported figures use deposit-kernel time unless explicitly labeled otherwise.

VIII. Results

A. Deposit-Kernel Runtime

Figure 1 shows deposit-kernel runtime versus N_p at 128^2 resolution with SoA layout and sorted particles. At $N_p = 10^5$, NGP completes in 1.99 ms, CIC in 0.59 ms, TSC in 0.90 ms, and Esirkepov in 4.33 ms. At $N_p = 10^6$, Esirkepov reaches 52.4 ms compared with 6.81 ms for CIC. CIC is the fastest scheme at moderate N_p ; Esirkepov is roughly 7 $\times$ slower than CIC after sparse trajectory optimization.

B. Memory Bandwidth

Figure 2 reports effective bandwidth versus grid size at $N_p = 10^5$. CIC achieves approximately 12.4 GB/s on the 128^2 grid; NGP is lower (~ 2.5 GB/s) because it performs fewer writes per particle. TSC bandwidth is comparable to CIC (~ 12.6 GB/s) despite nine writes, reflecting similar memory traffic at this particle density. Esirkepov reports under 1.7 GB/s because trajectory weight computation adds per-particle overhead beyond scatter bandwidth.

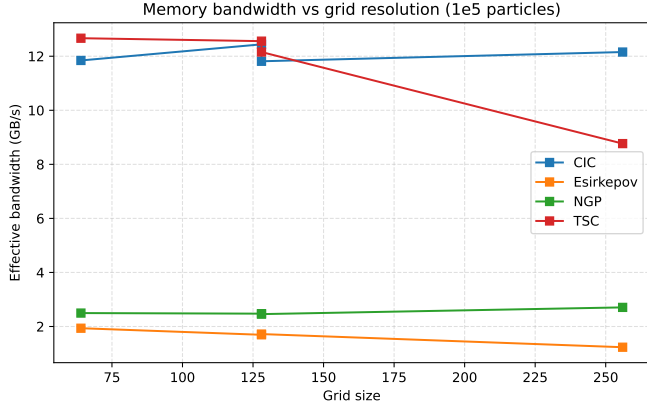


Fig. 2. Effective memory bandwidth vs. grid size (10^5 particles).

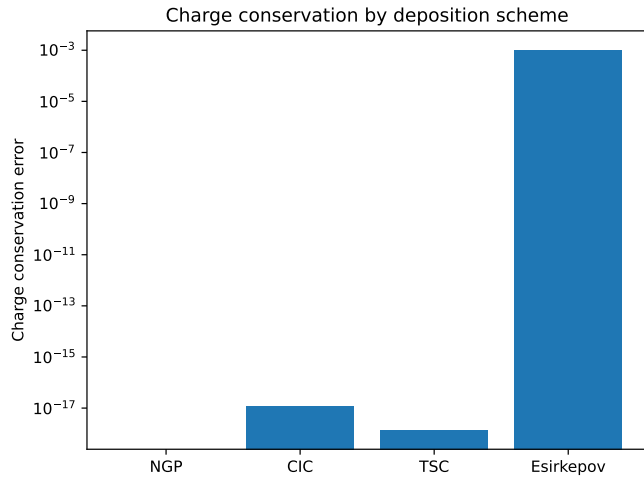


Fig. 3. Charge conservation error by deposition scheme.

C. Charge Conservation

Figure 3 and Table III report ϵ_Q after a single deposition pass. NGP achieves zero normalized error in the static test because each particle deposits its full charge to one node. CIC and TSC remain at machine precision ($\lesssim 10^{-17}$). Esirkepov shows higher single-step error ($\sim 10^{-3}$) in static microbenchmarks with finite Δt on a charge-neutral Maxwellian, but maintains low drift in full 500-step PIC cycles after the corrected deposit-before-push ordering.

D. Energy Drift

Figure 4 shows relative total-energy drift after 500 leapfrog steps. NGP exhibits the largest drift ($\sim -6.6 \times 10^{-6}$), consistent with its high grid noise. CIC and TSC remain at $\sim 10^{-5}$. Esirkepov drift improves to $\sim -3.0 \times 10^{-3}$ with corrected timestep ordering, remaining larger than CIC due to the interaction of trajectory deposition with the leapfrog field solve.

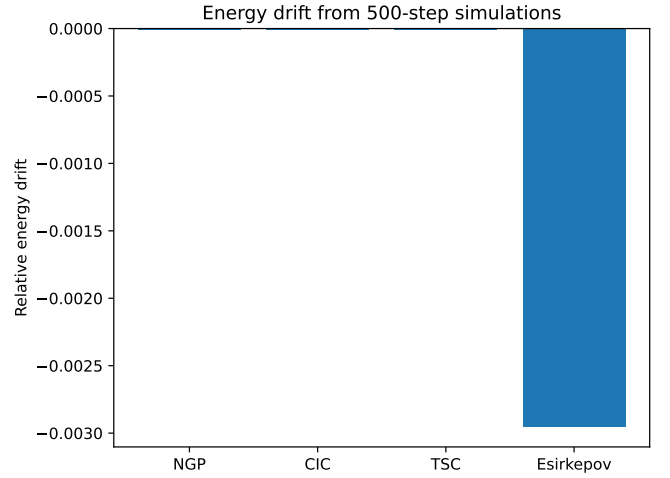


Fig. 4. Relative energy drift from 500-step simulations.

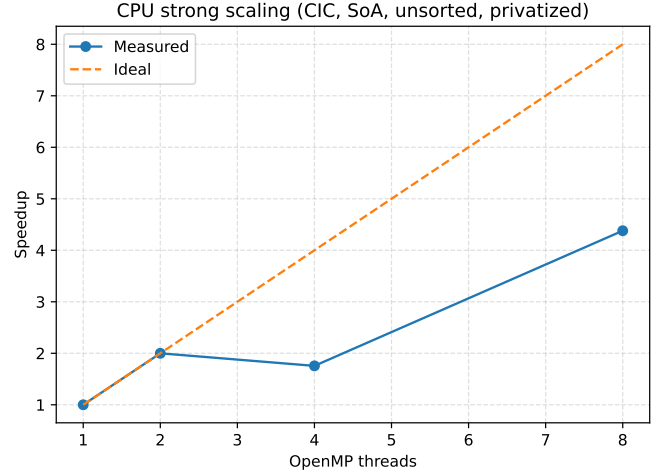


Fig. 5. OpenMP strong scaling for CIC deposition with privatized buffers.

E. OpenMP Strong Scaling

Figure 5 shows strong scaling for CIC with SoA layout and no sorting. Deposit time drops from 0.77 ms at $T=1$ to 0.38 ms at $T=2$, 0.44 ms at $T=4$, and 0.18 ms at $T=8$, yielding a $4.4\times$ speedup. Throughput gains flatten beyond two threads because privatized-grid merge and memory traffic dominate.

F. Spectral Noise

Figure 6 quantifies the high- k power fraction of deposited ρ for a uniform Maxwellian. NGP noise ratio is 0.61, CIC 0.39, TSC 0.26, and Esirkepov 0.39 in the static deposit test. TSC delivers the smoothest density field; NGP injects the most short-wavelength content.

G. Layout and Sorting Overhead

Figure 7 decomposes total deposition time into sort and deposit components for CIC with SoA layout. At

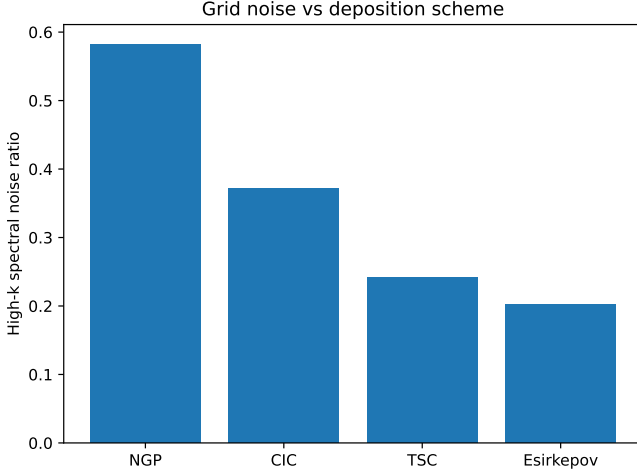


Fig. 6. High- k spectral noise ratio of deposited charge density.

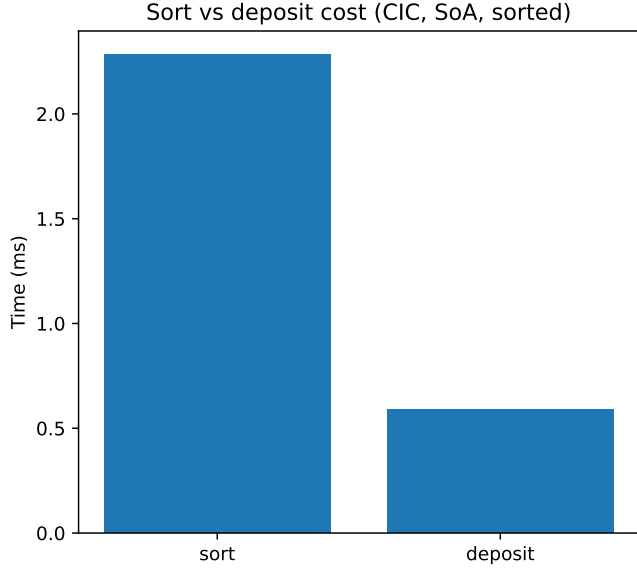


Fig. 7. Sort vs. deposit time for CIC with SoA layout.

$N_p = 10^5$, sorting costs 2.3ms while the deposit kernel takes 0.59ms. SoA and AoS achieve comparable deposit-kernel throughput; SoA sorting is more expensive because reordering requires a gather pass over separate attribute arrays.

Table III summarizes representative CPU results.

H. Langmuir-Wave Validation

All four schemes reproduce the same dominant oscillation frequency $\omega_{\text{meas}} = 0.122$ within 5% of one another, confirming scheme-independent linear physics in this test. The ratio $\omega_{\text{meas}}/\omega_{p,\text{macro}} \approx 1.73$ is identical across NGP, CIC, TSC, and Esirkepov, indicating a systematic offset from the 1D cold-plasma estimate rather than scheme-dependent error. TSC achieves the lowest spectral noise

TABLE III
Representative CPU deposition results ($N_p = 10^5$, 128^2 grid, SoA, sorted).

Scheme	Sort (ms)	Deposit (ms)	Thrpt. (10^8 p/s)	ϵ_Q
NGP	2.398	1.992	0.50	0.00e+00
CIC	2.282	0.589	1.70	1.16e-17
TSC	2.377	0.902	1.11	1.36e-18
Esirkepov	2.332	4.329	0.23	1.03e-03

(0.26) while preserving the same measured frequency as CIC.

IX. Discussion

A. Scheme Selection

NGP minimizes deposit-kernel time but exhibits the highest spectral noise (0.61) and largest 500-step energy drift. CIC provides the best accuracy–performance balance on CPU: fastest deposit-kernel time at moderate N_p , noise reduced by 36%, and energy drift an order of magnitude lower than NGP. TSC further reduces noise to 0.26 at roughly $1.5\times$ the CIC kernel time. Esirkepov maintains trajectory-based charge conservation in full PIC cycles but remains $\sim 7\times$ slower than CIC at $N_p = 10^5$ due to per-particle weight computation along crossed cells.

B. Memory Locality and Data Layout

Sorting improves spatial locality by clustering particles that share grid neighborhoods, but its $\mathcal{O}(N_p \log N_p)$ cost exceeds the deposit kernel at $N_p \leq 10^5$ when timed per deposition call. In full PIC cycles, sort cost can be amortized if particle order is preserved across steps or triggered periodically rather than every step. SoA and AoS achieve comparable deposit-kernel throughput on Apple Silicon; SoA’s advantage in unit-stride attribute reads is partially offset by the higher cost of SoA reordering. The choice of layout should therefore be guided by the full timestep profile, not the deposit kernel alone.

C. Parallelism Limits

OpenMP privatization removes atomic contention and yields a $4.4\times$ speedup from one to eight threads for CIC, but scaling flattens beyond two threads. Each thread allocates a full-grid private buffer ($T \times N_x N_y$ doubles) and merges through a critical section, so merge cost and memory bandwidth become the bottleneck. GPU atomics versus privatized tiles remain to be benchmarked on NVIDIA hardware where higher memory bandwidth and many lightweight threads may shift this balance.

X. Conclusion

We presented a unified, reproducible framework for evaluating charge deposition in 2D electrostatic PIC simulations. The codebase implements four deposition schemes behind a common runtime interface, separates sort from

deposit timing, and couples performance benchmarks with charge, energy, spectral-noise, and Langmuir-wave validation. On Apple Silicon, CIC offers the strongest accuracy–performance trade-off for CPU production runs; layout choice depends on whether sort overhead or attribute-access patterns dominate the full timestep. Esirkepov is preferable when trajectory-based charge conservation is paramount and particle counts are moderate. NGP is viable only when raw kernel speed outweighs noise tolerance.

XI. Future Work

Planned extensions include:

- CUDA benchmarking of atomic and privatized-tile kernels on Linux clusters with cuFFT-coupled Poisson solves.
- MPI domain decomposition with ghost-cell exchange to scale beyond shared memory.
- 3D extension with face-aligned sorting and wider stencils.
- Electromagnetic PIC with charge-conserving current deposition [11], [12].
- Incremental and GPU-resident sorting to reduce per-step reorder overhead.
- Absolute Langmuir-frequency calibration with corrected 2D plasma dispersion.

References

- [1] C. K. Birdsall and A. B. Langdon, *Plasma Physics via Computer Simulation*. IOP Publishing, 1991.
- [2] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*. CRC Press, 1988.
- [3] J. M. Dawson, “Particle simulation of plasmas,” *Reviews of Modern Physics*, vol. 55, no. 2, pp. 403–447, 1983.
- [4] D. Viereck et al., “Particle-in-cell algorithms for emerging computer architectures,” *Computer Physics Communications*, vol. 198, pp. 159–166, 2016.
- [5] B. Langdon, “Effects of the spatial grid in simulation plasmas,” *Journal of Computational Physics*, vol. 6, no. 2, pp. 247–268, 1970.
- [6] T. Z. Esirkepov, “Exact charge conservation scheme for particle-in-cell simulation with an arbitrary form-factor cloud,” *Computer Physics Communications*, vol. 135, no. 2, pp. 144–153, 2001.
- [7] A. Harvey et al., “Implementation of a charge conserving deposition scheme on hybrid parallel systems,” *Journal of Computational Physics*, vol. 299, pp. 229–240, 2015.
- [8] J. U. Brackbill and D. C. Barnes, “The effect of non-zero D product on the numerical solution of the magnetohydrodynamic equations,” *Journal of Computational Physics*, vol. 35, no. 3, pp. 426–430, 1980.
- [9] H. Okuda and J. M. Dawson, “Numerical simulation of the electron beam-plasma interaction,” *Physics of Fluids*, vol. 15, no. 5, pp. 921–931, 1972.
- [10] J. W. Eastwood, “The virtual particle electromagnetic particle-mesh method,” *Computer Physics Communications*, vol. 24, no. 1, pp. 73–84, 1981.
- [11] J. J. Wilke et al., “Particle-in-cell algorithms for GPU-accelerated supercomputers,” in *Proceedings of the Platform for Advanced Scientific Computing Conference*, 2017.
- [12] V. K. Decyk, “A modified PIC method for low-frequency electrostatic phenomena,” *Journal of Computational Physics*, vol. 107, no. 2, pp. 343–348, 1996.